PEEC-AI-SKILLS · V0.3

A closed growth loop for AI search visibility.

Production-tested Claude Code skills that turn a Peec AI project into operator-ready setup, diagnosis, and one-decision orchestration.

AUTHOR
Antonio Blago · Neuro-SEO System®

REPO
github.com/AntonioBlago/peec-ai-skills

DATE
2026-04-23

LLMs answer questions about your brand. You don't see those answers.

ChatGPT, Perplexity, Google AI Overview — they all recommend brands inside answers, not links. Whoever the LLM cites wins. Most teams have no idea what's being said.

~70%
of B2B buyers now use
AI assistants in research

0
of those queries
show up in Google Search Console

∅
SEO tooling that ties
AI mentions to actions

The gap nobody owns

Brand visibility tracking exists (Peec AI). Content tools exist. SEO data exists. But nothing connects "*my brand is invisible on prompt X*" to "*here's the one thing to do this week*".

Peec AI gives you the data layer.

Structured tracking of brand mentions inside LLM answers across ChatGPT, Perplexity, and Google AI Overview — with the actual scraped chats and their cited sources.

What Peec measures

- **Visibility per prompt** — is your brand cited in the answer?
- **Source URLs** — which pages did the LLM pull from?
- **Competitor share-of-voice** — who else gets mentioned?
- **Per-engine breakdown** — ChatGPT vs Perplexity vs GAO
- **Recommended actions** — `get_actions(scope=editorial|ugc|owned)`

…but data is not action

- 20+ prompts → **which one matters most**?
- Hero prompt at 0% visibility → **fix on-page or pitch a citation**?
- 3 competitors ranking → **study which one's content**?
- Should I write content, do outreach, or fix taxonomy?

> **peec-ai-skills** sits on top of Peec and answers exactly that question — every week, on demand, with one decision.

A closed growth loop — not a feature list.

Six stages, each owned by one skill. The orchestrator picks one move per cycle. SkillMind lifts lessons from project A into priors for project B.

```
ENTER       peec-start  (manual)             hooks/peec-detect.py (auto on session)
                                ↓
UNDERSTAND  peec-setup              competitors · prompts · funnel · taxonomy
                                ↓
DIAGNOSE    peec-checkup                     setup health · brand snapshot · top moves
                                ↓
ANALYZE     peec-cluster          peec-content-intel (sources)
                                ↓
DECIDE      peec-agent  (orchestrator — calls /peec-checkup first)
                                ↓
EXECUTE     content-write (Visibly skill)    peec-outreach
                                ↓
LEARN       peec-report             attribution · next moves
                                ↓
            feeds back into UNDERSTAND ────────────────────┐
                                                           ↓
CROSS-PROJECT MEMORY     peec-learn   read priors · write patterns after lift
```

All skills are user-invocable in Claude Code. Hooks auto-detect Peec context and inject state into the model.

9 skills, one entry point, zero ambiguity.

Each skill owns one job. You never wonder which one to call — `/peec-start` dispatches based on state.

Skill	Role	What it does
`/peec-start`	ENTER	Single entry point. Reads state, decides next skill.
`/peec-setup`	UNDERSTAND	9-phase setup: competitors, prompts, taxonomy. Phase 0: full / import / audit / partial / skip.
`/peec-checkup`	DIAGNOSE	Read-only health pass. Inventory + setup quality + brand snapshot + top 5–8 improvements.
`/peec-cluster`	ANALYZE	Turns prompts into 4–8 strategic content zones with attack vector per zone.
`/peec-content-intel`	ANALYZE	Per-prompt brief: Query Fan-Out + competitor scoring + forum pain mining.
`/peec-agent`	DECIDE	Orchestrator. Always invokes /peec-checkup first, then picks ONE 4-week move.
`/peec-outreach`	EXECUTE	Outreach pipeline from get_actions + UGC discovery. 5 pitches/week cap.
`/peec-report`	LEARN	Weekly closer: ≤400-word narrative + 3 next-actions + 1 stop-doing.
`/peec-learn`	MEMORY	Lifts patterns out of one project into priors for the next.

/peec-start — one safe entry point.

Reads `setup_state.json`, optionally probes Peec live, dispatches to the right downstream skill. Never produces deliverables itself — pure routing.

```
/peec-start
   │
   ├── no state, Peec empty       →  /peec-setup  (full)
   ├── no state, Peec populated   →  /peec-setup  (import — brownfield)
   ├── state present, < 90 days   →  /peec-agent      (decide one move)
   ├── state present, > 90 days   →  /peec-setup  (audit — drift check)
   └── state has missing phases   →  /peec-setup  partial:<phase>
```

Why this matters

First-time users on the orchestrator hit a hard stop ("no setup state, run setup first"). `/peec-start` closes that loop — even on a Peec project that someone else already configured, it does the right thing without re-running discovery.

/peec-checkup — the read-only health pass.

One report, four sections. Works from day 1 of Peec data. Never writes to Peec, never modifies state. The lens you and the orchestrator both look through.

§0 Inventory

- Brands · Prompts (per funnel stage) · Topics · Tags · Chats analysed
- Window + days_with_data — explicit "insufficient data" badge if <7

§1 Setup health (P0/P1/P2 score)

- SaaS-tool brands as competitors? → P0
- Funnel stage < 20% of prompts? → P1
- Tags only the default 4? → P1
- Brands with Umlauts but no ASCII alias? → P0

§2 Brand performance snapshot

- Visibility per stage / per engine
- Hero prompts winning vs losing (top 5 each)
- Source diversity — weak (<5) / OK / healthy (>15)
- Competitor delta on weakest stage

§3 Top 5–8 improvements (priority-ranked)

- Score = impact × actionability × strategic_fit
- Each item: causal trace + handoff skill + S/M/L effort + 4-week metric

/peec-agent — one decision, not a dashboard.

Always invokes /peec-checkup first as its diagnostic input. Then picks exactly one tier, one skill, one 4-week metric. Logs every decision so the next run reads the history.

Gap priority ladder

Tier	Gap
1	Data < 7 days → STOP
2	Setup health < 80% → setup audit
3	< 20 prompts → partial:prompts
4	No zones → cluster-builder
5	Hero loses → content-write
6	Source diversity < 5 → outreach
7	4+ weeks running → reporter

Higher tier always wins. One per cycle.

Decision schema (excerpt)

```
Build the content zone "KI-SEO Retainer Decision"
by 2026-04-26 with one hero article.

Why: 47 Peec prompts active, but Decision stage
only has 3 weak prompts (2 at 0%). Retainer is
the main revenue product — Decision must be
strongest, currently weakest.

How:  /content-write
      project_id:    or_bf5b...
      focus_keyword: "KI-SEO Retainer"
      page_type:     blog
      language:      de  (from setup_state)
      country:       DE  (from setup_state)

Measurable in 4 weeks:
  Visibility on pr_a38381d4: 0% → ≥15%
  Zone visibility:           → >10%

Not doing now:
  · outreach (no own asset to pitch yet)
  · cluster refresh (zones still valid)
```

/peec-setup — 9 phases, full-funnel coverage.

Takes a Peec project from empty (or broken) to operator-ready. Owns `setup_state.json` — every other skill refuses to run without it.

When to invoke

- "Set up Peec for <client>"
- "My Peec competitors are wrong"
- "Design prompts for my customer journey"
- "Restructure topics / tags"
- Audit of an existing tracking setup

Scope modes

```
scope=full                  # greenfield, all 9 phases
scope=audit                 # live-diff, re-run drifted phases
scope=partial:gsc_mapping   # jump to one phase
scope=import                # brownfield reconstruction
```

The 9 phases

1. **Phase 0** — state check & mode selection
2. **Competitor discovery** from real AI chats (not Google)
3. **Customer-journey prompts**: Awareness → Consideration → Decision → Retention
4. **Topic taxonomy** (one topic per prompt, funnel-aligned)
5. **Tag taxonomy** (cross-cut: branded/non-branded/informational/transactional/…)
6. **GSC keyword mapping** via Visibly
7. **Forum pain-point mining** (Reddit, Gutefrage, t3n, OMR)
8. **Hero prompt** definition
9. **P0/P1/P2 backlog** + state write

> **Never silently defaults to EN/en.** Country + language travel through every downstream skill via `setup_state.json` .

/peec-cluster — zones, not keyword groups.

Reduces a flat prompt set to 4–8 **strategic zones**, each with one structural weakness in competition and one measurable metric. Output is a strategic map — not a content calendar.

A zone is valid only when all three axes line up

```
      Intent
       ↑
Visibility gap
       ↑
Demand signal
```

If a candidate zone fails any axis → merged or dropped.

When to invoke

- ≥20 Peec prompts — team has lost the overview
- Quarterly strategy refresh
- Before an investment conversation ("what is our AI content strategy?")

Do not use with <10 prompts or when funnel stages aren't set.

Output — one zone map + Peec tags

- **Markdown zone map** — one zone per section, each with: structural weakness · measurable metric · exact next action
- **Peec tag per zone** (`zone:<slug>`) — all prompts retagged so `/peec-report` can measure zone lift later

Per-zone anatomy

```
## Zone: KI-SEO Retainer Decision
Structural weakness:
  Competitors rank listicles with USP matrix,
  we have 1 case study, no comparison piece.
Metric (4 weeks):
  Zone visibility 3% → >10%
Next action:
  /peec-content-intel pr_a38381d4
  → brief → publish pillar
Prompts (6):  pr_a38381d4, pr_c2..., ...
Evidence:     forum quotes · SERP patterns
```

/peec-content-intel — one prompt in, one brief out.

For one Peec prompt the brand is losing: sub-queries (Query Fan-Out), verbatim buyer pains, competitor URL scoring, outline, focus keywords, outreach targets. Publish-ready.

Data sources (combined)

- **Peec** — prompt visibility, source URLs, scraped chats
- **Visibly AI** — backlinks, onpage, keywords, GSC, **Query Fan-Out**
- **Forum mining** — Reddit, Gutefrage, t3n, OMR (country-filtered)

When to invoke

- "Which content wins Peec prompt X?"
- "Analyze the sources competitors get cited for"
- "What's the gap between me and competitor.de?"

Requires ≥24h of Peec data on the target prompt.

Output — `briefs/YYYY-MM-DD_<slug>/`

```
brief.md                ← publish-ready brief
competitor-urls.json    ← scored attackability
forum-pains.json        ← verbatim quotes
scoring.json            ← raw Query Fan-Out scores
```

Query Fan-Out in practice

Your page "*antonioblago.de*" on seed *"SEO Freelancer"*:
Coverage **53.3%** · 8 of 15 sub-queries covered
7 concrete gaps: "Aufgabenbereiche eines SEO Freelancers" · "Welche SEO Tools nutzen Freelancer?" · "Preise und Leistungen Vergleich" …

Each gap maps 1:1 to an H2/H3 in the brief outline.

/peec-outreach — off-page pipeline, not a domain list.

Most AI-visibility lift comes from citations on domains LLMs already crawl. Turns `get_actions` + UGC discovery into a prioritized pipeline with concrete, target-specific pitches — capped at 5/week.

Four quadrants, one pipeline

Quadrant	Example	Deliverable
OWNED	Own site	Content update
EDITORIAL	t3n, OMR	Pitch + contact
REFERENCE	Wikipedia	Edit request
UGC	Reddit, Gutefrage	Answer draft

When to invoke

- After `/peec-content-intel` — opportunity list is in
- Weekly ritual: 3–5 quality pitches
- When `get_actions` surfaces high-opportunity rows

Per-target output

- Domain + contact (author, editor, admin)
- Pitch text, tonality-matched to target
- Concrete quote + concrete offer (case study, data, expert quote)
- Status tracker: `queued → sent → replied → cited | rejected`

Measurement loop (4 weeks)

```
Week 0  baseline: citations from Peec
Week 1  send 5 pitches
Week 2  send 5 pitches
Week 3  send 5 pitches
Week 4  re-pull Peec → citations delta

Attribution summary feeds into
/peec-report as a scored
outreach channel.
```

Tracker lives at `<project>/outreach/YYYY-Wkk_outreach_log.md`.

/peec-report — 400 words beats 15 charts.

Closes the feedback loop every week. Three questions per cycle, answered in ≤400 words — plus a `learnings.json` that feeds the next orchestrator run as priors.

Three questions, one narrative

1. **What moved?** — visibility trend per prompt / cluster / zone
2. **Why?** — which specific investment caused which lift
3. **What next?** — 3 prioritized actions + 1 stop-doing

Fifteen charts don't get read. 400 words do.

Cadence

- **Weekly** — active projects with running content + outreach
- **Monthly** — retainer projects in maintenance
- **Quarterly** — strategy review — feeds next cluster run

Output — `<project>/growth_loop/YYYY-MM-DD_report.md`

```
## What moved
Brand visibility 8% → 12% in 14d.
Decision stage 3% → 9% (+200%).
Awareness flat at 24%.

## Why
+ Published pillar "KI-SEO Retainer"  → +6%
+ Reddit answer on r/selbststaendig   → +3 citations
- t3n editorial pitch: no reply

## Next
1. Clone pillar pattern for Awareness hero
2. Double down on Reddit (weekly cadence)
3. STOP t3n pitches — audience fit low
```

> `learnings.json` → consumed by next `/peec-cluster` & `/peec-outreach` run as priors.

/peec-learn — cross-project memory layer.

Project-local learnings live in `growth_loop/learnings.json`. This skill **promotes** them across projects: lessons from project A become priors for project B.

Two modes

- **Write** — extract 1–3 patterns from a just-produced artifact (decision, brief, zone map, outreach log, learnings.json) and persist to SkillMind.
- **Read** — on orchestrator start, recall patterns matching gap type / project / skill → hand back as priors.

Pattern quality bar

- **Causal** — X caused Y, not correlation
- **Falsifiable** — states the condition for being wrong
- **Evidence-backed** — links the artifact that produced it

No speculation. Artifact must have a measured outcome.

Backend

> Pinecone-backed semantic memory via `mcp__skillmind__*`. Fallback when MCP unavailable: append to `<project>/growth_loop/patterns.md`.

Example pattern (write mode)

```
{
  "title":
    "DE B2B: UGC beats editorial 8× on AI citations",
  "tags": ["peec", "DE", "outreach", "UGC"],
  "summary":
    "On DE B2B SEO projects, Reddit + Gutefrage
     answers produced 40% reply rate and +3
     citations per week. Editorial pitches to
     t3n/OMR: 5% reply, 0 citations in 4 weeks.",
  "provenance": {
    "project_id": "or_bf5b...",
    "source_skill": "peec-report",
    "date": "2026-04-23"
  }
}
```

On next project, `/peec-outreach` recalls this as a prior before building the pipeline.

Auto-detection: hooks inject state silently.

Two hooks wrap `hooks/peec-detect.py`. Silent unless the prompt mentions Peec keywords or a setup_state.json is found in CWD. Never pollutes non-Peec sessions.

SessionStart hook

- Walks CWD + 2 parent levels for `growth_loop/setup_state.json`
- If found → injects state summary + recommended skill
- If not found → silent (exit 0)

UserPromptSubmit hook

- Reads stdin JSON, extracts prompt
- Matches against keyword list (peec, sichtbarkeit, ai visibility, ...)
- If match + no state → injects "`/peec-start`" hint
- Otherwise silent

JSON envelope per Claude Code hook schema

```
{
  "hookSpecificOutput": {
    "hookEventName": "SessionStart",
    "additionalContext":
      "=== PEEC PROJECT DETECTED ===\n
       State file: ...\n
       Project:    antonioblago.de\n
       Locale:     country=DE · language=de\n
       Setup age:  143 days (very stale)\n
       RECOMMENDED NEXT ACTION:
       /peec-setup audit\n
       ============================="
  }
}
```

> Claude Code sees this as system context and decides whether to act.

setup_state.json — the memory the loop turns on.

Single shared file at `<project>/growth_loop/setup_state.json`. `/peec-setup` writes it, all other skills read it — or refuse to run.

```
{
  "peec_project_id": "or_bf5b...",
  "domain": "antonioblago.de",
  "target_country": "DE",
  "prompt_language": "de",
  "secondary_languages": [],
  "setup_version": "1.1",
  "completed_at": "2026-04-23T14:30:00Z",
  "phases_completed": [
    "competitors", "prompts",
    "topics", "tags",
    "gsc_mapping", "forum_mining"
  ],
  "snapshot": {
    "brands": 8, "prompts": 24,
    "topics": 6, "tags": 18
  },
  "hero_prompt_id": "pr_a38381d4",
  "last_audit_at": null
}
```

Why this matters

- **No silent EN defaults.** Country + language are explicit and travel through every downstream skill.
- **Brownfield-aware.** Existing Peec project? Phase 0 detects it, infers `completed_at` from `min(brand.created_at, prompt.created_at)`, and writes state without touching Peec.
- **Age-driven re-runs.** <30d skip · 30–90d audit · >90d full re-run.
- **One source of truth.** No drift between agent, hooks, dashboard.

> Schema + protocol: `skills/_shared/SETUP_STATE.md`

v1.2 schema — business type & audience as first-class inputs.

`peec-setup` Phase 0.5 elicits three fields once and persists them. Every downstream content skill reads them. No more generic briefs.

Three new fields in `setup_state.json`

```
{
  "setup_version": "1.2",
  "business_type": "b2b-service",
  "audience": {
    "primary":        "Shop-Owner DACH 3-20 MA, Shopify",
    "buyer_personas": ["Shop-Owner Shopify DACH",
                       "SEO-fatigued E-Commerce Decider"],
    "pain_points":    ["verbrennt Budget",
                       "3 Agenturen gewechselt"]
  },
  "page_type_taxonomy": [
    "pillar", "landing_page", "blog_post",
    "case_study", "comparison", "faq", "pricing"
  ]
}
```

6 allowed `business_type` values · `page_type_taxonomy` auto-generated from it (see next slide).

Phase 0.5 — ask once, persist once

- **Never guessed.** A `.de` shoe-shop is not a `b2b-service` — ask explicitly.
- **One turn.** Business type + audience + pain-points in one elicitation.
- **Migration.** v1.1 state triggers Phase 0.5 on next `/peec-setup audit`.

Downstream consumers

`/peec-content-intel`	`page_type` required per brief
`/peec-cluster`	zone one-move names a `page_type`
`/peec-agent`	`pain_points` as Awareness priors
`/peec-report`	attributes lift per `page_type`

> Wrong `business_type` corrupts every brief. Ask once, ask early.

Business-type → page-type matrix.

Every content move names a page type. The allowed set is not your preference — it is a constraint derived from what actually wins for your business model in AI answers.

business_type	Allowed `page_type` values	Typical winning combo
b2b-service freelancer, agency	`pillar` · `landing_page` · `blog_post` · `case_study` · `comparison` · `faq` · `pricing`	Decision → `landing_page` + `case_study`
b2c-ecommerce D2C, Shopify	`pdp` · `collection` · `pillar` · `blog_post` · `guide` · `category_page` · `faq`	Decision → `pdp` · Consideration → `collection`
b2b-saas subscription SW	`landing_page` · `integration` · `use_case` · `blog_post` · `comparison` · `docs` · `pricing`	Decision → `use_case` · Retention → `docs`
info-product courses, memberships	`sales_page` · `webinar_lp` · `blog_post` · `case_study` · `faq` · `lead_magnet`	Decision → `sales_page`
local-service catchment-area	`local_landing` · `landing_page` · `case_study` · `blog_post` · `faq`	Decision → `local_landing`
marketplace multi-seller	`collection` · `pdp` · `category_page` · `pillar` · `blog_post`	Most queries → `collection`

Auto-fallback rule

When a zone's top competitors cite a URL class your `business_type` cannot produce (e.g. LISTICLE for `b2b-service`), `/peec-cluster` **switches the one-move from content to outreach**. No content is written that can't win. If *every* zone ends up on outreach → taxonomy mismatch P0, re-run Phase 0.5.

PART 2 — TUTORIAL

From zero to first decision in under 15 minutes.

Install the skills. Connect Peec. Run /peec-start on your existing project.
The orchestrator picks the first move — you ship it.

Prerequisites.

Two required, two optional. Skills degrade gracefully when optional pieces are missing.

Tool	Required?	Why	Install
Claude Code	Required	The harness that runs the skills (CLI, VS Code, JetBrains, web)	claude.com/claude-code
Peec AI MCP	Required	Provides `mcp__peec-ai__*` tools — the data layer	OAuth (see next slide)
Visibly AI MCP	Optional	GSC / GA4, backlinks, onpage, Query Fan-Out (≥v0.6.0)	HTTP MCP (zero install)
SkillMind MCP	Optional	Cross-project memory for peec-learn	`pip install skillmind`

Graceful degradation

Without Visibly: GSC mapping + Query Fan-Out skipped, briefs still produced.
Without SkillMind: cross-project priors fall back to `growth_loop/patterns.md`.
Without Peec MCP: nothing works — this is the data spine.

Install in 30 seconds.

One bash script symlinks all 9 skills into `~/.claude/skills/`. `git pull` in the repo updates every installed skill in place.

Default install (symlink)

```
git clone https://github.com/AntonioBlago/peec-ai-skills.git ~/peec-ai-skills
cd ~/peec-ai-skills
./claude-peec-ai.sh                              # symlink all 9 skills
```

Useful flags

```
./claude-peec-ai.sh --copy                       # copy instead of symlink (no git pull updates)
./claude-peec-ai.sh --target ./.claude            # per-project install
./claude-peec-ai.sh --force                       # overwrite existing skill dirs
./claude-peec-ai.sh --dry-run                     # preview, no fs changes
./claude-peec-ai.sh --uninstall                   # remove all skill entries
./claude-peec-ai.sh --only peec-start,peec-checkup   # partial install
```

> On Windows: symlinks need Developer Mode or admin rights. Use `--copy` as a safe fallback.

Connect the Peec MCP — OAuth, no API key.

Single command. First tool call opens browser to authorize. Tokens persist; subsequent runs are silent.

CLI

```
claude mcp add peec-ai --transport streamable-http https://api.peec.ai/mcp
```

Or directly in `~/.claude/settings.json` / `~/.claude.json`

```
{
  "mcpServers": {
    "peec-ai": {
      "type": "http",
      "url": "https://api.peec.ai/mcp"
    }
  }
}
```

Optional: Visibly AI MCP (recommended)

```
{
  "visiblyai": {
    "type": "http",
    "url": "https://mcp.visibly-ai.com/mcp",
    "headers": { "Authorization": "Bearer lc_your_key" }
  }
}
```

First run on an existing Peec project.

No destructive 9-phase setup. `/peec-start` detects the brownfield, asks 3 questions, writes state with the *real* setup date.

```
User       /peec-start
Skill      Reads <cwd>/growth_loop/setup_state.json — missing.
           Probes Peec live: list_brands, list_prompts, list_topics, list_tags.
           → Detected: 8 brands, 24 prompts, 6 topics, 18 tags.

Skill      "Detected existing Peec setup. Import as state, or run full setup?
            Target country (ISO)?    Prompt language (ISO)?"
User       import · DE · de

Skill      Infers completed_at = min(brand.created_at, prompt.created_at)
           → 2025-09-12  (~ 224 days ago)
           Writes growth_loop/setup_state.json (atomic .tmp + rename).

Skill      "State imported. Inferred setup date: 2025-09-12 (~224 days ago).
            → Setup is > 90 days. Recommended: /peec-setup audit."
```

The orchestrator never lies about setup age. If your Peec project is 6 months old, the state reflects that — and the next move is an audit, not a "fresh setup" report.

Phase 0 mode selection — what actually runs.

Cheap (one file read + at most one parallel Peec read). Decides whether the rest of the setup is needed at all.

State file	Live Peec content	Mode	What runs
Missing	Empty (≤2 brands AND ≤4 prompts AND ≤0 topics)	full	All 9 phases. Greenfield.
Missing	Populated (≥3 brands OR ≥5 prompts OR ≥1 topic)	import	Reconstruct state, ask 3 questions, exit. No discovery.
Present, < 30 days	—	skip	Show summary, ASK before re-running.
Present, 30–90 days	—	audit	Live-diff snapshot. Re-run only phases with >20% drift.
Present, > 90 days	—	full (warn)	Recommend full re-run; data is stale.

User can always force a mode

```
/peec-setup scope=full              # ignore state, re-run from scratch
/peec-setup scope=audit             # force audit even on fresh state
/peec-setup scope=partial:gsc_mapping   # jump to one specific phase
```

/peec-checkup output anatomy.

Saved to `<project>/checkups/YYYY-MM-DD_checkup.md`. Mirrored to stdout. The orchestrator reads this same file as its input.

```
# Peec Checkup — antonioblago.de · 2026-04-23

## TL;DR
- Visibility now: 12% (was 8% 14d ago)
- Setup health:   78% (1 P0, 3 P1)
- Strongest funnel: Awareness (24%) · Weakest: Decision (3%)
- Top competitor on weak stage: rauschsinnig.de
- Recommended next action: /peec-cluster

## §0 Inventory                  ## §1 Setup health (78%)            ## §2 Brand performance
| Bucket          | Count |       P0 — Brand "Stürkat" missing       Visibility per stage:
| Brands tracked  |   8   |       ASCII alias                          Awareness    24%
| Prompts total   |  24   |                                            Consideration 18%
|  · Awareness    |   6   |       P1 — Decision stage only             Decision      3%   ← weakest
|  · Decision     |   3   |       3 prompts (12.5% of total)           Retention    14%
|  · ...                          P1 — Tags = default 4 only         Hero prompts losing (top 5):
| Chats analysed  | 184   |                                          1. "KI-SEO Retainer für DACH" — 0%
| Window          | 28d   |                                            top: rauschsinnig.de @ 38%

## §3 Top 5 improvements (priority-ranked, score = impact × actionability × strategic_fit)
#1   Build content zone "KI-SEO Retainer Decision"    →  /peec-cluster   M    +12% zone vis. 4w
#2   Fix Decision-stage prompt mass (3 → 7 prompts)   →  /peec-setup       S    funnel 3% → 8%
#3   Pitch own case-study to t3n.de + omr.com         →  /peec-outreach         M    source div +3
#4   Add "Stuerkat" alias to brand                    →  /peec-setup       S    chat match fix
#5   Brief: pr_a38381d4 (KI-SEO Retainer für DACH)    →  /peec-content-intel        L    pr visibility +20%

Recommended next action: /peec-cluster — because Decision stage has the biggest
visibility gap AND highest actionability score.
```

/peec-agent flow — checkup-first orchestration.

No duplicate diagnostic logic. The agent runs the checkup, reads its sections, picks one tier, hands off with parameters pre-filled, logs the decision.

```
Step 0    Pre-flight: read setup_state.json — STOP if missing or incomplete.

Step 1a   /peec-checkup   →   <project>/checkups/2026-04-23_checkup.md  (always run)
Step 1b   Read aux history: *_learnings.json, decisions_log.md, outreach_log.md
Step 1c   Optional: mcp__skillmind__recall(query, k=5, filter_tags=["peec"])

Step 2    Detect gap from checkup §0+§1+§2 (priority ladder, higher tier wins)

Step 3    Produce decision  (one verb, one object, one 4-week metric)

Step 4    Hand off with parameters pre-filled  (execute, don't recommend)

Step 5    Append to growth_loop/decisions_log.md
          ## 2026-04-23 — peec-cluster
          Checkup ref: checkups/2026-04-23_checkup.md
          Signal:      Decision stage 3% — biggest gap (checkup §2)
          Tier picked: 4   ·  Override of detected tier? no
          Decision:    Build "KI-SEO Retainer Decision" zone
          ...
```

Execute · Learn · Remember.

The four skills that close the loop. Each has one job; together they convert one decision into measurable lift, then turn that lift into a transferable pattern.

Skill	What it produces	Cadence
`/peec-content-intel`	Per-prompt content brief: gap URLs + Query Fan-Out + forum pains + competitor breakdown + outline + outreach targets. Saves to `briefs/<date>_<slug>/`.	Per losing hero prompt, on demand
`/peec-cluster`	4–8 strategic content zones (intent × funnel × gap × demand). Persists zones as Peec `zone:*` tags for later attribution.	When prompts ≥ 20
`/peec-outreach`	Outreach pipeline: target table + per-target pitch templates + tracker file. Cap 5 pitches/week. After 4 weeks: attribution summary.	Weekly ritual
`/peec-report`	≤400-word narrative: what moved · why · 3 next-actions + 1 stop-doing. Saves `growth_loop/<date>_report.md` + `learnings.json`.	Weekly (active) / monthly (maintenance)
`/peec-learn`	After any measurable outcome: extracts 1–3 transferable patterns (causal, falsifiable, evidence-backed) and persists to SkillMind. Next orchestrator run recalls them as priors.	After reporter closes a cycle

GET STARTED

Closed loop — not a dashboard.

9 production-tested skills, one entry point, persistent state, automatic detection. Free, MIT-licensed, no SaaS. Bring your own Claude Code.

INSTALL + READ + CONTRIBUTE

github.com/AntonioBlago/peec-ai-skills

AUTHOR
Antonio Blago

EMAIL
info@antonioblago.com

PROFILE
antonioblago.com